

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 3

Выполнила:

Савченко Анастасия

Группа К3341, Поток БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать Dockerfile для каждого микросервиса приложения бронирования столиков, написать общий docker-compose.yml, настроить сетевое взаимодействие между сервисами.

Ход работы

1. Подготовка структуры проекта

За основу взят код микросервисов из ЛР2. Каждый сервис (auth-service, restaurant-service, booking-service, api-gateway) дополнен Dockerfile. Для каждого сервиса создан файл Dockerfile, описывающий сборку контейнера на базе Node.js 20 Alpine.

2. Dockerfile для каждого сервиса

Все сервисы используют общий подход к контейнеризации:

auth-service, restaurant-service, booking-service:

- Базовый образ: node:20-alpine
- Рабочая директория: /app
- Установка зависимостей через npm install
- Компиляция TypeScript через npx tsc
- Порт: 8001, 8002, 8003 соответственно
- Команда запуска: node dist/index.js

api-gateway:

- Базовый образ: node:20-alpine
- Установка зависимостей через npm install
- Порт: 3000
- Команда запуска: npx tsx src/index.ts (без компиляции, для простоты проксирования)

3. Общий docker-compose.yml

Создан файл `docker-compose.yml`, описывающий все сервисы системы:

Сервис	Порт	Зависимости
postgres	5432	—
rabbitmq	5672, 15672	—
auth	8001	postgres (healthy)
restaurant	8002	postgres (healthy), rabbitmq
booking	8003	postgres (healthy), rabbitmq
gateway	3000	auth, restaurant, booking

Для PostgreSQL настроен healthcheck с проверкой `pg_isready`, что гарантирует готовность БД перед запуском зависимых сервисов. Для хранения данных используется том `pgdata`.

4. Настройка сетевого взаимодействия

Docker Compose автоматически создаёт внутреннюю сеть, в которой сервисы доступны друг другу по именам контейнеров. В коде выполнены следующие замены:

- В `data-source.ts` каждого сервиса `host: "localhost"` заменён на `host: process.env.DB_HOST || "localhost"`, что позволяет задавать хост через переменные окружения.

- В `docker-compose.yml` для сервисов `auth`, `restaurant`, `booking` установлена переменная `DB_HOST`: `postgres` — имя контейнера PostgreSQL.

- В `api-gateway` все URL заменены с `localhost` на имена контейнеров: `auth:8001`, `restaurant:8002`, `booking:8003`.

— В booking-service и restaurant-service для RabbitMQ localhost:5672 заменён на rabbitmq:5672.

5. Запуск и проверка

Система запущена командой: `docker compose up -d --build`

Все шесть контейнеров успешно стартовали. Проверена доступность API Gateway на порту 3000. Эндпоинты возвращают корректные данные из базы данных, заполненной через seed-скрипты.

6. Заполнение базы данных

Для заполнения БД тестовыми данными использовались seed-файлы. Для работы внутри контейнера в них заменён хост БД с localhost на postgres. Файлы скопированы в контейнеры и запущены командами:

```
docker cp auth-service/seed.ts lab3-auth-1:/app/seed.ts
```

```
docker exec -it lab3-auth-1 npx tsx seed.ts
```

```
docker cp restaurant-service/seed.ts lab3-restaurant-1:/app/seed.ts
```

```
docker exec -it lab3-restaurant-1 npx tsx seed.ts
```

После заполнения API возвращает список кухонь, ресторанов, работает создание бронирований и отзывов.

Вывод

В результате выполнения лабораторной работы приложение бронирования столиков полностью контейнеризовано. Для каждого из трёх микросервисов и API Gateway созданы Dockerfile. Написан общий docker-compose.yml, описывающий все сервисы, базу данных PostgreSQL и RabbitMQ. Настроено сетевое взаимодействие: сервисы общаются внутри Docker-сети по именам контейнеров. Система запускается одной командой `docker compose up`. Приложение готово к развёртыванию на удалённом сервере.